

TontineChain

Cahier des charges & pitch de soutenance

Tontine digitale pour l'Afrique de l'Ouest

FCFA (Mobile Money) · Trésor Bitcoin · WhatsApp · Preuves cryptographiques

Version document : juin 2026 — État du dépôt tontine-trust

1. Résumé exécutif

TontineChain modernise la tontine informelle (ROSCA) en Afrique de l'Ouest : cotisations en FCFA via Mobile Money (Kkiapay), règles automatisées, trésor collectif indexé sur Bitcoin, bot WhatsApp pour l'inclusion, et registre de preuves cryptographiques. Le projet vise une soutenances académique crédible : honnêteté sur ce qui est réel on-chain versus comptable en base de données.

Pitch (30 secondes)

"Nous digitalisons les tontines communautaires sans exclure ceux qui n'ont qu'un téléphone et Mobile Money. Les cotisations passent par Kkiapay, les règles sont figées avant activation, et un trésor Bitcoin mainnet vérifiable sur mempool.space renforce la confiance. Un bot WhatsApp permet de créer un groupe ou consulter son solde sans installer l'app. Chaque engagement peut être signé en secp256k1 et stocké en registre."

Proposition de valeur

- Inclusion : WhatsApp + MoMo, pas besoin de compte bancaire classique
- Transparence : tours, pénalités et statuts visibles en temps réel
- Sécurité collective : assurance vie obligatoire (référence NSIA démo)
- Réserve Bitcoin : dépôts réels on-chain + registre interne PostgreSQL
- Accessibilité : exploration publique (/rechercher, /crypto) sans compte

2. Périmètre fonctionnel livré

2.1 Module Tontine (ROSCA)

- Création de groupe : montant, fréquence, ordre (aléatoire / VRF / manuel), pénalités
- Invitation par lien / code (REJOINDRE TONT-XXXX via WhatsApp)
- Catalogue public : 4 groupes démo + politique RLS groupes ouverts
- Activation du premier tour (rpc_activate_group)
- Suivi membres : payé / en attente / retard
- Assurance vie obligatoire : numéro police NSIA pré-rempli (NSIA-VIE-2026-0042817)
- Déclaration décès membre → procédure assurance (UI + messaging)

2.2 Paiements FCFA (Kkiapay)

- Portefeuille interne FCFA (profiles.wallet_balance)
- Cotisations via Kkiapay sandbox (widget React)
- Webhook Kkiapay (edge function) + triggers PostgreSQL
- Mise à jour automatique du pool groupe et statut membre
- Crédit trésor : 2 % de chaque cotisation réussie → registre btc_ledger

2.3 Trésor Bitcoin (modèle honnête)

Deux volets distincts, clairement séparés dans l'UI et le code :

- On-chain : dépôts BTC réels sur adresse trésor bc1q... (mainnet), sync mempool.space
- Registre interne : conversions FCFA→sats, stake, crédits 2 % cotisations (btc_ledger)
- Achat sats : rpc_btc_buy_fcfa refuse si liquidité on-chain insuffisante
- Tables : btc_treasury_pool, btc_on_chain_txs, btc_ledger, btc_user_wallets

- Edge function : btc-treasury-sync (déployée sur Supabase cloud)
- Cours live : CoinGecko (XOF/BTC)
- Page /crypto : sync dépôts, réconciliation mempool / base / registre

2.4 Preuves & engagements

- Signatures ECDSA secp256k1 (viem) — même courbe que Bitcoin, hors chaîne
- Table bitcoin_commitments (hash, signature, pubkey_hint)
- Engagement tontinier signable depuis GroupeDetail et /crypto
- Formulation honnête : preuve d'engagement, pas transaction Bitcoin

2.5 Bot WhatsApp

- Numéro démo : +229 01 43 26 71 02
- Commandes : CREER, REJOINDRE, TONTINE, MEMBRES, /solde, /cotiser, /bitcoin, /liquidity, /score
- Module unifié : supabase/functions/_shared/whatsapp/
- Dev local : vite-plugin-whatsapp + scripts PowerShell de test
- Prod : edge function whatsapp-webhook + tontine-automation (cron tours)

2.6 Application web (PWA)

- React 18 + Vite 8 + TypeScript + Tailwind + shadcn/ui
- Design system DESIGN.md (Stripe + Linear + accent Bitcoin)
- Layout desktop (max-w-7xl), landing marketing, mockups iPhone
- Auth Supabase, score confiance, notifications, admin garanties
- 22 tests Vitest, build production OK

3. Architecture technique

Couche	Technologie
Frontend	React, Vite, Tailwind, PWA
Backend	Supabase PostgreSQL + Auth + RLS
Edge Functions	whatsapp-webhook, btc-treasury-sync, tontine-automation
Paielements	Kkiapay (sandbox MoMo UEMOA)
Bitcoin	mempool.space, CoinGecko, registre SQL
Crypto preuves	viem secp256k1 (off-chain)
Hébergement	Supabase cloud (slyizcavccnkvxqtmfmd)

3.1 Schéma Bitcoin (invariants)

sats_liquid = BTC réellement disponible pour convertir FCFA en sats. internal_sats = allocations comptables (achats + 2 % cotisations). on_chain_sats = total ingéré depuis mempool. Les cotisations MoMo ne créent pas de BTC on-chain automatiquement.

3.2 Routes principales

- Publiques : /, /crypto, /whatsapp, /rechercher, /rejoindre/:id, /connexion
- Authentifiées : /home, /creer, /groupe/:id, /cotiser, /portefeuille, /score, /admin

4. Scénario de démo soutenance (~10 min)

#	Action	Point jury
1	Landing /	Valeur : tontine + Bitcoin + MoMo + WhatsApp
2	/crypto	Cours BTC, trésor 3 colonnes, sync dépôts (si adresse configurée)
3	/whatsapp	AIDE, CREER, /bitcoin, /solde
4	Inscription → /home	Tableau de bord desktop, portefeuille FCFA
5	/rechercher	Catalogue groupes publics sans compte
6	Créer groupe → Activer	Assurance NSIA, règles figées, preuves
7	/cotiser	Flux Kkiapay sandbox
8	/groupe/:id	Membres, registre, 2 % trésor après cotisation

5. Discours jury — ce qu'il faut assumer

Ce qui est réel

- Paiements MoMo via Kkiapay (sandbox en démo)
- Dépôts BTC sur adresse trésor vérifiables sur mempool.space
- Journal immuable btc_ledger + triggers PostgreSQL
- Signatures cryptographiques secp256k1 stockées en base

Ce qui est simulé / comptable

- 2 % cotisation → sats internes sans achat BTC automatique
- Assurance NSIA : référence enregistrée, pas d'API partenaire
- Preuves : pas d'ancrage OP_RETURN Bitcoin
- APY trésor : chiffre statique, non calculé dynamiquement
- Portefeuille membre : identifiant registre:..., pas adresse BTC personnelle

6. Reste à faire (backlog)

6.1 Bloquant pour démo Bitcoin complète

- Configurer VITE_BTC_TREASURY_ADDRESS (bc1q mainnet) dans .env
- Secrets Supabase : BTC_TREASURY_ADDRESS, BTC_NETWORK=mainnet
- Envoyer un petit dépôt BTC réel → /crypto → Sync dépôts
- Script : .\scripts\setup-btc-treasury.ps1 -Address "bc1q..."

6.2 Production & sécurité

- Restreindre RLS btc_treasury_pool (UPDATE réservé service_role)
- Kkiapay production (clés live, webhook signé)
- WhatsApp Meta : tokens prod, webhook vérifié
- Cron tontine-automation planifié (CRON_SECRET)
- Audit secrets (.env jamais commité)

6.3 Fonctionnel

- RPC + UI unstake (schéma prévu, non exposé)
- Réconciliation stricte btc_user_wallets ↔ btc_ledger
- API assurance partenaire (vérification police NSIA)
- Retrait BTC on-chain vers membre (hors scope MVP)
- Corriger wording « preuve Bitcoin » → « signature secp256k1 » partout
- Recalculer ou retirer APY / TVL marketing si jury pointilleux

6.4 Qualité & déploiement

- supabase db push aligné migrations locales / cloud
- CI GitHub (test + build automatiques)
- Déploiement frontend (Vercel / Netlify) + domaine
- Documentation utilisateur FR (guide WhatsApp)

7. Indicateurs de maturité (estimation)

Domaine	État	Note indicative
Tontine ROSCA	MVP fonctionnel	16/20
Paielements MoMo	Sandbox OK	14/20
Bitcoin honnêteté	Architecture solide	15/20
Bitcoin on-chain	En attente adresse user	12/20
WhatsApp bot	Dev + edge déployable	14/20
UI / Design	Landing + app desktop	16/20
Sécurité prod	RLS à durcir	11/20

8. Livrables du dépôt

- Code source : github.com (tontine-trust)
- Migrations SQL : supabase/migrations/ (12 fichiers)
- Edge functions : whatsapp-webhook, btc-treasury-sync, tontine-automation
- Tests : src/test/ (22 tests)
- Design : DESIGN.md
- Scripts : deploy-cloud.ps1, setup-btc-treasury.ps1, test WhatsApp

Document genere automatiquement a partir de l'etat du projet TontineChain (juin 2026). Projet academique / demonstration - usage selon les conditions du depot.